# Anteater Software Spec Manual

Version 0.1

Authors: Prithvi Binu, Justin Wu, Katrina Gong, William Culley, Stefano Pinna Segovia, Rishab Senthil Kumar

Affiliation: University of California, Irvine
Team Name: Blunder Boys + Katrina

---

# Table of Contents

Table of Contents....2
**Glossary.... 3**
**1. Software Architecture Overview.... 5**
1.1 Main data types and structures....5
1.2 Major software components....5
1.3 Module interfaces.... 6
1.4 Overall program control flow....6
1. Initialize board setup....6
1. Main control flow (main loop)....6
a. Setup.... 6
b. display the board.... 6
c. Repeat.... 6
**2. Installation.... 7**
2.1 System requirements and compatibility....7
2.2 Setup and configuration....7
2.3 Building, compilation, and installation....7
**3. Documentation of Packages, Modules, and Interfaces....7**
3.1 Detailed description of data structures.... 7
3.2 Detailed description of functions and parameters....8
3.3 Detailed description of input and output formats.... 9
**4. Development Plan and Timeline....9**
4.1 Partitioning of Tasks....9
4.2 Team member responsibilities.... 9
**Copyright.... 10**
**Error Messages.... 10**
**Index.... 10**
**References....11**

# Glossary

| Term | Definition |
|---|---|
| Ant | Another name for a Pawn which is the basic piece that can only move one tile forward unless it has not moved yet. In that case, it would be able to move 2 tiles forward. An Ant is able to promote into any other piece (other than another Ant or King), when it reaches the opposite side of the chessboard. |
| Ant eating | A special move by the Anteater that can capture multiple Ants at once. The move is initiated by the Anteater capturing the first Ant. If there are any adjacent tiles with Ants, it will target that direction until there are no Ants left, provided that they are at the same rank or file. |
| Bishop | A piece that moves diagonally across the board any number of spaces. |
| Castling | A special move that can be played once by each player. This move allows the king to move left or right two tiles to swap places with the Rook on the kingside or queenside. Both the Rook and the King may not have moved a single tile before the attempt to castle. No pieces may be in between them while castling. |
| Check | When a player's King is vulnerable to capture on the opponent's next turn. |
| Checkmate | When a player's King is under check and cannot escape. |
| En Passant | A special pawn capture move that means "in passing". This move starts when an Ant moves two tiles forward from its initial position to attempt to avoid capture from an enemy Ant. The enemy Ant is allowed to capture it as if it had only moved one tile forward. |
| File | The vertical columns on a chessboard, labeled alphabetically (A-J). |

| | |
|---|---|
| King | The most important piece and target for attack. Moves one square in any direction. The game ends when the king is in checkmate. |
| Knight | A piece that moves in an L-shape, two squares in one direction and one square perpendicular. The only piece that can jump over other pieces. |
| Legal moves | The list of moves allowed for a piece. The King's condition is relevant. If a player's king is under check, a move that is played must get the King out of that condition. |
| Pawn | A piece that moves forward one square, or two squares from its starting position, and may only capture diagonally. Also called an ant. |
| Promotion | When a pawn reaches the opposite end of the board, it is promoted to another piece |
| Queen | The most powerful piece, moves in any direction, horizontally, vertically, and diagonally any number of square. |
| Piece | Items that are moved on a chessboard, each with their unique movesets (King, Queen, Rook, Bishop, Knight, Ant, Anteater). |
| Rank | The horizontal rows on a chessboard, labeled numerically (1-8). |
| Rook | A piece that moves horizontally or vertically any number of squares. |
| Stalemate | When a player's King is not under check, but they are not able to make a legal move, resulting in a draw. |

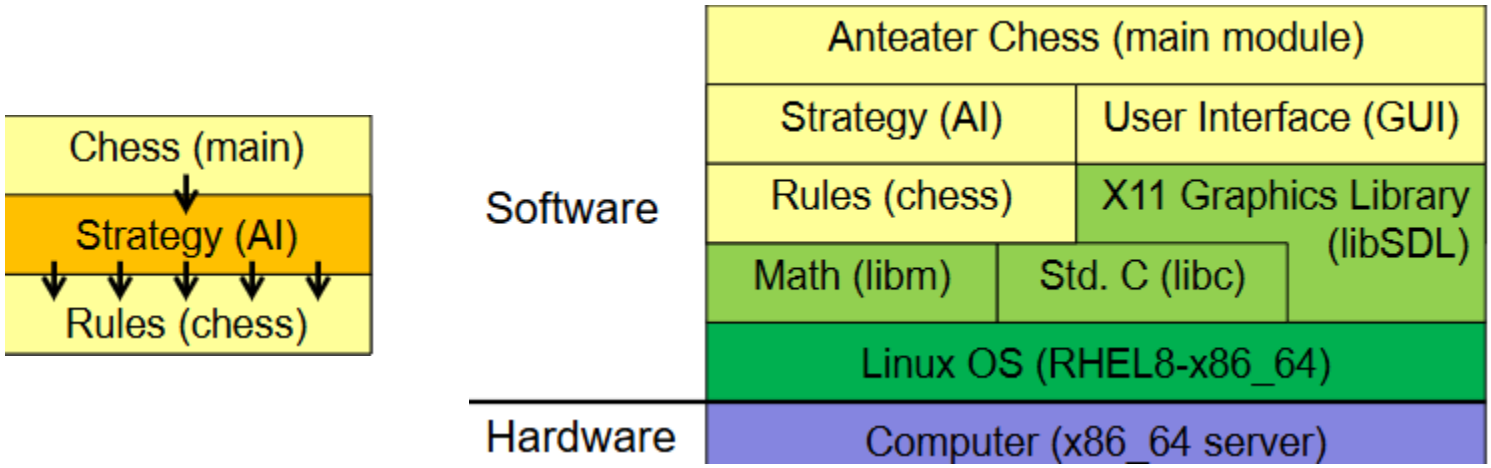
# 1. Software Architecture Overview

## 1.1 Main data types and structures

1. Struct Player -
    a. Contains: name, color, isHuman
2. typedef enum Piece type
    a. Contains: ant, knight, bishop, rooke, queen, king, and anteater
3. typedef enum Piece color
    a. Contains: white and black
4. Piece - struct *
    a. Contains: piece type and piece color enums
5. struct piece * array Board
    a. Contains: 2D w/ NUM_RANKS=8 & NUM_FIL ES=10
6. Position of piece - struct *
    a. Contains: Rank = char, File =  char
7. Struct location
    a. Contains: Rank = int, File = int
8. Move - struct
    a. Pos1, pos 2
9. Log of moves - struct { int move, int piece, int specialMoves, history[1000]}
    a. Contains:
        i. Moves - char[]
        ii. Piece - struct piece w/ pieceColor and pieceType
        iii. moveNumber - int
        iv. moveHistory - char[]
10. Struct GameState
    a. Stores the following:
        i. Board
        ii. currentPlayer
        iii. enPassantTiles
        iv. CastlingRights
        v. halfMove_count ←for 50 move draw rule
        vi. fullMove_count
        vii. Log of moves
    b. This struct is useful for tracking special rules like castling or En Passant, and storing the board and log of moves.
    c. Functions that make changes to the board should interact with the GameState by reference (gameState*).
    d. Functions that only access the GameState's attributes should use const reference(const gameState*).

## 1.2 Major software components

- Diagram of module hierarchy




## 1.3 Module interfaces

- API of major module functions
- Interact with the GUI by picking up the pieces on the board and dragging them to the desired position
- Undo button on the interface to allow player to undo their move and return to the previous position
- Labels stating which player's turn it is
- Buttons for player to select which color to play at the start

## 1.4 Overall program control flow

1. Initialize board setup
    a. Setup all objects including board, pieces, and players
    b. Display board using GUI
2. Repeat following steps
    a. Prompt User for Move
    b. Check for legality
    c. White player makes a move
    d. Display the board
    e. If black is in checkmate, white wins!
    f. Black player makes a move
    g. Display the board
    h. If white is in checkmate, black wins

# 2. Installation

## 2.1 System requirements and compatibility

Operating System: RHEL8 64-bit Linux

## 2.2 Setup and configuration

1. Download the user package from the server
   >> git clone team16@zuma.eecs.uci.edu:repos/chess.git
2. Extract the user package using the following commands:
   % gtar xvzf BinaryArchive.tar.gz
   % evince chess/doc/chess.pdf
   % chess/bin/chess

- The above commands allow the user to extract the chess folder using [gtar] and view the documentation using the document viewer with [evince]. It will also execute the chess program.

## 2.3 Building, compilation, and installation

1. Run makefile to compile and build all modules
   a. Chess game is built based on installed modules

# 3. Documentation of Packages, Modules, and Interfaces

## 3.1 Detailed description of data structures

```
struct player {
    char name[];
    char color[];
    bool isHuman;
}
typedef enum pieceType {
    ant,
    knight,
    bishop,
    rook,
```

```
    queen,
    king,
    anteater
};
struct Piece{
    enum PiecsType Piece;
    enum Piececolor color;
 }
struct piece *Board [8][10];
struct pos { // use for move
    char file; // a-j
    int rank; // 1-8
};
struct location { // use to log moves
    int rank; // 1-10
    int file; // 1-8
};
struct move {
    char pos1[]; // "a4"
    char pos2[]; // "a5"
};
struct log {
    char move[]; // "a1 - j10"
    struct Piece piece; // n = knight, q = queen, etc
    int moveNumber; // 1-1000 for # of moves in the game
    char history[1000]; // Ne4, knight to e4
};
struct GameState {
    struct piece *Board [8][10];
    struct player currentPlayer;
    struct enPassantTiles epT;
    struct CastlingRights CR;
    int halfMove_count;
    int fullMove_count;
    struct log logOfMoves;
}
```

## 3.2 Detailed description of functions and parameters

- Function prototypes and brief explanations
- struct Move* getMoves(int* moveCount)
    - Detects which player's turn it is and then returns all legal moves the player can do as a pointer to an array of moves. It should do this by looping through each piece that the current player controls and attempting to move that piece to every other tile on the board. The move is validated by a function called isLegalMove. If isLegalMove is true, then append the move to the array.
- struct Move* getLegalMoves(struct piece p, int* moveCount)
    - Returns all legal moves only for the specific piece
    - Uses isLegalMove to determine if all of the possible moves from getMoves are legal
- bool isLegalMove(struct Move, const struct gameState* gs)
    - Finds the piece on the board at the start position of the move struct and then checks if the piece belongs to the current player. Based on the piece, it should be able to determine which moves are legal (maybe using a switch-case). For example, it should return true if the absolute distance the king moves vertically or horizontally is <= 1. Note that a function may need to check if the King would be in check as a result.
- int getScore(const struct gameState* gs)
    - Attempts to get the score of all the pieces owned by the current player. It bases this off the chart below. This is a helper function for the miniMax function. The miniMax attempts to maximize the score of the white pieces and minimize the score of the black pieces.
    - Proposed Values for Evaluation Function:
    - Pawn = 100
    - Knight = 300
    - Bishop = 350
    - Rook = 500
    - Queen = 900
    - Anteater= 330 (placeholder)
- int miniMax(const struct gameState* gs, int depth, int alpha, int beta, bool playerColor)
    - A function that recursively evaluates all possible moves the computer can play, while getting the end score of each move. If the current player is white, miniMax tries to maximize the score, and if it's black, it tries to minimize the score. The return value would be the score.
    - The depth parameter determines how much branching is required. For example, a depth of 3 would mean the computer moves, then the player, and then the computer again. A lower depth would mean less branching and less

total time taken per decision, with the tradeoff of potentially picking a worse move.
    - Alpha and beta are used for an optimization of the miniMax algorithm called alpha-beta pruning.
- int movePiece_Computer(struct gameState* gs)
    - This function repeatedly attempts legal moves and computes the end score with the miniMax function. It tracks the highest end score if white and the lowest end score if black, and plays the associated move that caused that score.
- movePiece(struct piece p, struct move m)
    - Generic move function to move any given piece to a legal position
    - Uses isLegalMove to check validity of the move desired
- void convertPos (struct pos position)
    - Used to convert the rank and file of a piece into a single integer to represent its location on the board following array conventions
    - Used to input move into the board array
- char[] convertLogMove (struct log Log)
    - Grabs move from log history to convert into integer
    - Used for undo functionality
- void logMove (struct pos position, struct piece Piece, struct log logOfMoves)
    - Takes the rank and file of the new position of a chess piece and logs it into the log object
    - Takes the piece type and color to put into the log object as well
- Int checkmate (struct gameState* gs)
    - Checks whether the either opponent is in check or checkmate and returns an integer based on that

### 3.3 Detailed description of input and output formats

- Syntax or format of a move entered by the user
    - char[] “a2”, char []“a3”
- Syntax or format of a move recorded in the log file
    - char move[] = “a2, a3”, piece.color = black, piece.type = ant,
    - moveNumber = 2, history = {“aa2, aa3”}
- Human readable log file using chess notation
    - ex: “aa2, aa3”

# 4. Development Plan and Timeline

### 4.1 Partitioning of Tasks

• main program

• user interface (textual and/or graphics)
• chess objects (data structures for pieces, boards, moves)
• chess rules (possible moves, legal moves)
• lists (or trees) of moves
• strategy (artificial intelligence, AI) module
• log file module
• feature modules
• documentation and testing (everyone do)

### 4.2 Team member responsibilities

Will - Chess Rules, Main program
Stefano - List of moves, GTK (GUI)
Katrina - Pieces moveset (+ legal moves)
Justin - log file module, Chess objects
Prithvi - feature modules, error handling
Rishab - strategy (artificial intelligence, AI) module

Timeline:
Testing and debugging due 4/19
Version 0.5 due 4/20
Version 1.0 due 4/28

# Copyright

© 2026 Blunder Boys + Katrina

# Error Messages

| Error | Meaning |
|---|---|
| Illegal Move. Please try again. | The entered move violates the rules of chess. Check piece movement and game rules and try again. |
| Wait for opponent move | Out of turn move |
| Time warning | Tells how much time left |
| King is in Check! | Check warning |
| Cannot castle while in check.<br>King or rook has already moved. | Caslting warnings |

| | |
|---|---|
| Pieces are blocking castling path. | |
| En passant not available. | Can capture pawn like it has moved one move even if it moved two if the enemy pawn lands next to your pawn after moving two squares in first move. |
| | |

# Index


Ant (Pawn) 3
Ant eating 3
Bishop 3, 4, 8
Castling 3, 5
Check 3, 4, 8
Checkmate 3, 4, 6, 7
En passant 3, 5
King 8
Knight 4, 8
Pawn 3, 4, 8
Queen 4, 5, 8
Rook 3, 4, 8


# References